Классификация изображений с использованием нейронных сетей

Кибенко Дарья (К3344)
ФПиН, ИТГС, 3 курс

Цель практики

Разработка модели нейронной сети для автоматической классификации светильников по стилям

Задачи практики

Изучение предметной области

I задача

III задача

Обучение модели

Сбор датасета

II задача

IV задача

Тренировка модели на наборе данных

Главные используемые технологии

Python

Популярный, универсальный язык программирования с простым синтаксисом. Широко используется в анализе данных, машинном обучении и веб-разработке.

Tensorflow

Открытая библиотека от Google для машинного обучения. Позволяет создавать и обучать нейронные сети с высокой производительностью.

01 СБОР И ФОРМИРОВАНИЕ ДАТАСЕТА

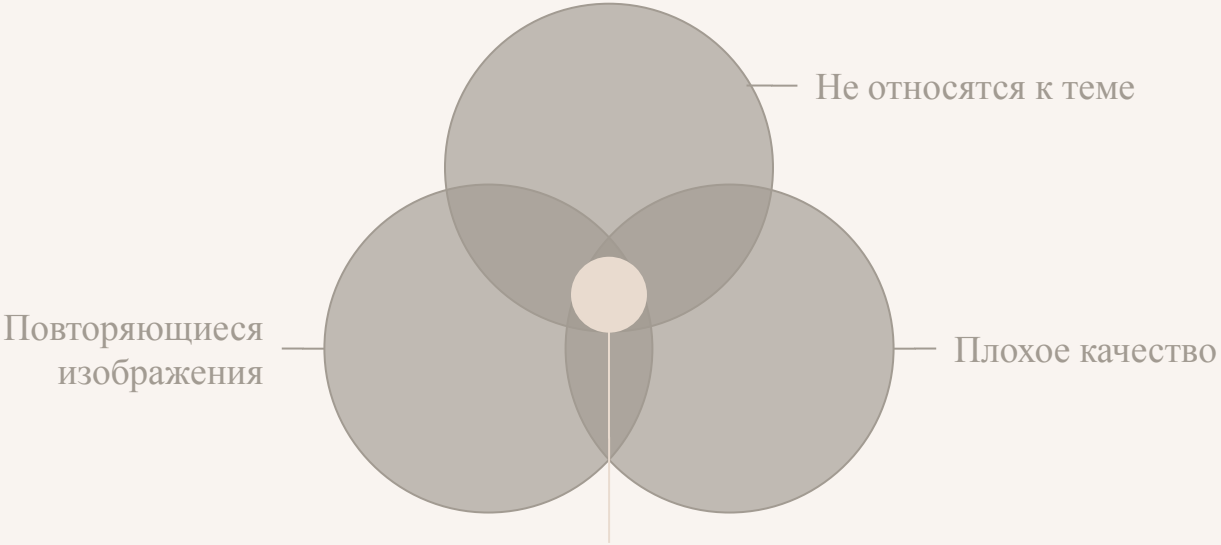
02 РАЗРАБОТКА МОДЕЛИ И ОБУЧЕНИЕ НА ОДНОЙ ПОЛОВИНЕ

03 ДОРАБОТКА КОДА И ОБУЧЕНИЕ НА ОБЪЕДИНЕННОМ НАБОРЕ ДАННЫХ

01 ДАТАСЕТ

Сбор данных и формирование их в наборы

Датасет: формирование и удаление лишнего



Уникальные

02

РАЗРАБОТКА

Создание кода и обучение модели

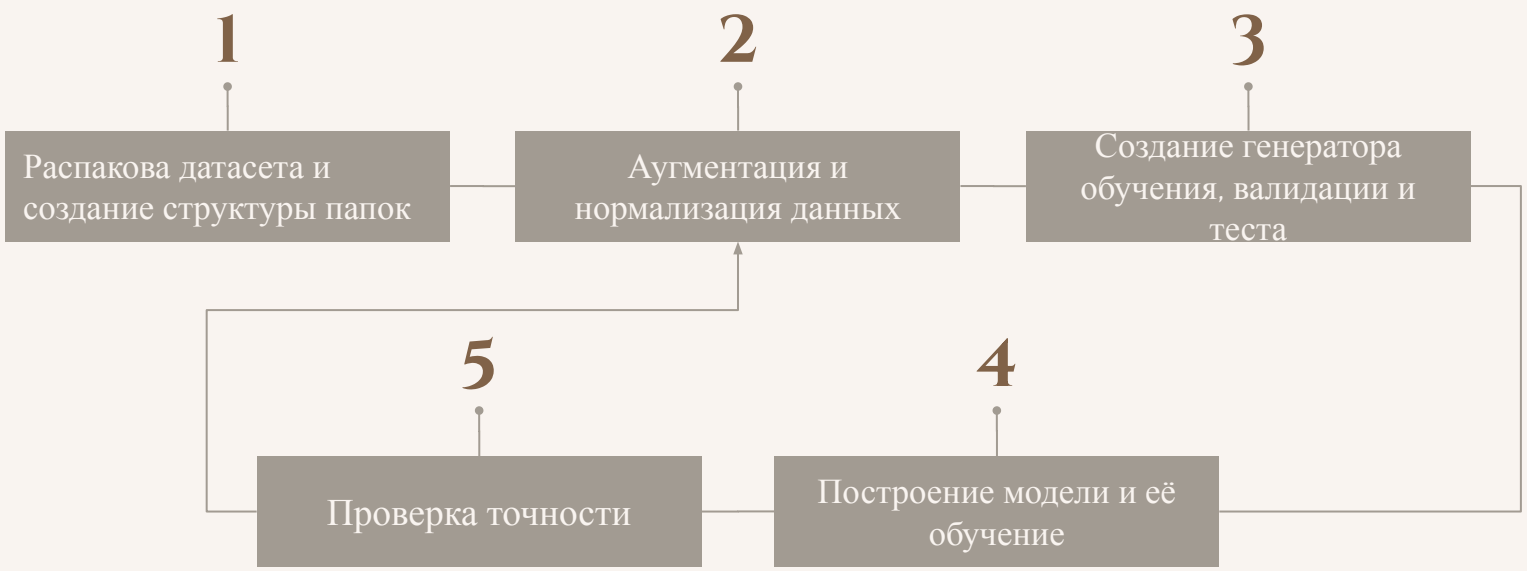
Что будет происходить?

Проект направлен на классификацию изображений люстр по стилям. Для первых тренировочных данных были выбраны стили: классика, модерн и ретро.

70%

Достичь определённое количество процентов точности на тестовом наборе данных

Алгоритм


1
Распакова датасета и создание структуры папок
2
Аугментация и нормализация данных
3
Создание генератора обучения, валидации и теста
4
Построение модели и её обучение
5
Проверка точности

Разделение данных

TEST	VALIDATE	TRAIN
10%	20%	70%
Даёт финальную оценку модели.	Оценивает качество в процессе обучения.	Используется для обучения модели

Этапы работы кода и модели

Фаза 1

Подготовка

Загрузка и распаковка ZIP-архива.
Создание директории с подкатегориями.

Фаза 2

Обработка

Генерация аугментированных изображений с помощью *albumentations*.
Обработка изображений через *ImageDataGenerator*.

Фаза 3

Модель

Модель *MobileNetV2*, обученная с *categorical_crossentropy*.

Фаза 4

Результат

Валидация и вывод метрик.

```python
ZIP_FILE = "chandeliers.zip"
```

```python
EXTRACT_DIR = "raw_images"
```

```python
DATASET_DIR = "chandeliers_dataset"
```

```python
CATEGORIES = ["classic_chandlier", "moder_chandelier", "retro_chandlier"]
```

```python
SPLIT = ["Train", "Test", "Validate"]
```

```python
# --- Константы ---
ZIP_FILE = "chandeliers_dataset.zip"
EXTRACT_DIR = "raw_images"
DATASET_DIR = "chandeliers_dataset"
CATEGORIES = ["classic_chandelier", "modern_chandelier", "retro_chandelier"]
SPLITS = ["Train", "Validate", "Test"]

# Распаковка ZIP
if not os.path.exists(EXTRACT_DIR):
    os.makedirs(EXTRACT_DIR)
with zipfile.ZipFile(ZIP_FILE, "r") as zip_ref:
    zip_ref.extractall(EXTRACT_DIR)

print("✅ Архив успешно распакован!")
```

```python
# --- Создание папок ---
for split in SPLITS:
    for category in CATEGORIES:
        os.makedirs(os.path.join(DATASET_DIR, split, category), exist_ok=True)

# --- Функция разбиения данных ---
def split_dataset(source_folder, train_ratio=0.7, val_ratio=0.2, test_ratio=0.1):
    images = [img for img in os.listdir(source_folder) if img.lower().endswith(('.jpg', '.png', '.jpeg'))]
    random.shuffle(images)
    train_split = int(len(images) * train_ratio)
    val_split = int(len(images) * (train_ratio + val_ratio))
    return images[:train_split], images[train_split:val_split], images[val_split:]

# --- Распределение изображений по папкам ---
for category in CATEGORIES:
    source_folder = os.path.join(EXTRACT_DIR, category)
    if not os.path.exists(source_folder):
        print(f"⚠️ Папка {source_folder} не найдена, пропускаем.")
        continue
    train_images, val_images, test_images = split_dataset(source_folder)
    for img_name in train_images:
        shutil.copy(os.path.join(source_folder, img_name), os.path.join(DATASET_DIR, "Train", category, img_name))
    for img_name in val_images:
        shutil.copy(os.path.join(source_folder, img_name), os.path.join(DATASET_DIR, "Validate", category, img_name))
    for img_name in test_images:
        shutil.copy(os.path.join(source_folder, img_name), os.path.join(DATASET_DIR, "Test", category, img_name))
```

```
[ ] # --- Генераторы изображений ---
    IMAGE_SIZE = (300, 300)  # Увеличиваем размер для лучшего распознавания деталей
    BATCH_SIZE = 32

    train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=30,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.3,
        horizontal_flip=True,
        vertical_flip=True,
        fill_mode='nearest'
    )

    val_datagen = ImageDataGenerator(rescale=1./255)
    test_datagen = ImageDataGenerator(rescale=1./255)
```

```
[ ] # --- Генераторы данных ---
    train_generator = train_datagen.flow_from_directory(
        os.path.join(DATASET_DIR, "Train"),
        target_size=IMAGE_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='categorical'
    )

    val_generator = val_datagen.flow_from_directory(
        os.path.join(DATASET_DIR, "Validate"),
        target_size=IMAGE_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='categorical'
    )

    test_generator = test_datagen.flow_from_directory(
        os.path.join(DATASET_DIR, "Test"),
        target_size=IMAGE_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        shuffle=False
    )
```

```
[ ] # --- Создание модели ---
    base_model = EfficientNetB3(weights='imagenet', include_top=False, input_shape=(300, 300, 3))

    # Заморозим базовые слои для первого этапа обучения
    for layer in base_model.layers:
        layer.trainable = False

    # Добавляем дополнительные слои
    x = Flatten()(base_model.output)
    x = Dense(1024, activation='relu')(x)
    x = Dropout(0.5)(x)
    x = Dense(512, activation='relu')(x)
    output = Dense(len(CATEGORIES), activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=output)

    # --- Компиляция модели ---
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.0001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
```

```python
# --- Колбэки ---
early_stopping = keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, min_lr=1e-6)

# --- Обучение ---
EPOCHS = 30
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=EPOCHS,
    class_weight=class_weights,
    callbacks=[early_stopping, reduce_lr]
)

# --- Разморозка последних слоёв для дообучения ---
for layer in base_model.layers[-30:]:
    layer.trainable = True

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.00001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# --- Дообучение ---
history_fine = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=15,
    class_weight=class_weights,
    callbacks=[early_stopping, reduce_lr]
)
```

```
[ ] # --- Оценка точности ---
    test_loss, test_accuracy = model.evaluate(test_generator)
    print(f"✅ Точность на тестовых данных: {test_accuracy * 100:.2f}%")
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 25s 9s/step - accuracy: 0.7097 - loss: 0.8354
✅ Точность на тестовых данных: 56.45%
```

Улучшение кода

```python
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomRotation(0.3),
    tf.keras.layers.RandomZoom(0.3),
    tf.keras.layers.RandomContrast(0.3),
    tf.keras.layers.RandomBrightness(0.2),
])
```

```python
base_model = tf.keras.applications.MobileNetV2(input_shape=IMG_SIZE + (3,),
                                               include_top=False,
                                               weights="imagenet")
base_model.trainable = False  # Замораживаем базовые слои

# Добавляем дополнительные слои
model = tf.keras.Sequential([
    data_augmentation,
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.5),  # Dropout для предотвращения переобучения
    tf.keras.layers.Dense(3, activation="softmax")  # 3 класса: classic, modern, retro
])
```

```python
EPOCHS = 100

history = model.fit(train_ds,
                    validation_data=val_ds,
                    epochs=EPOCHS)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 81ms/step - accuracy: 0.5496 - loss: 1.0929
🔥 Итоговая точность на тестовом наборе: 55.88%
```

03

Разработка 2.0

Объединение данных и апгрейд кода

Финальные исправления

Перешли только *MobileNetV2* — выше точность при тех же ресурсах.

Внедрены ***EarlyStopping*** и ***ReduceLROnPlateau*** — стабилизация обучения.

Оптимизирован импорт, добавлены проверки путей, исключены дубли.

Добавлено ***mixed precision training*** — ускорение обучения на GPU.

ПОСЛЕДНИЕ ПРАВКИ

:

Моменты из кода

Архитектура модели 1

Веса классов 2

Исправление №1

```
def create_mobilenetv2():
    base_model = MobileNetV2(weights="imagenet", include_top=False, input_shape=(224,
224, 3))
    for layer in base_model.layers[:-20]:
        layer.trainable = False
    x = GlobalAveragePooling2D()(base_model.output)
    x = Dense(512, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.4)(x)
    x = Dense(256, activation='relu')(x)
    x = BatchNormalization()(x)
    x = Dropout(0.4)(x)
    output = Dense(len(CATEGORIES), activation='softmax', dtype='float32')(x)

    return Model(inputs=base_model.input, outputs=output )
```

АРХИКТЕТУРА МОДЕЛИ

3 Callbacks

4 Mixed precision

Моменты из кода

Архитектура модели 1

Веса классов 2

⋮

Исправление №1

```
model = create_mobilenetv2()
model.compile(optimizer=initial_optimizer, loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

3 Callbacks

4 Mixed precision

АРХИКТЕТУРА МОДЕЛИ

Моменты из кода

Архитектура модели 1

Веса классов 2

Исправление №2

```
for layer in model.layers:
    layer.trainable = True
model.compile(optimizer=fine_tune_optimizer,
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(train_dataset,
validation_data=val_dataset, epochs=15,
class_weight=class_weights,
callbacks=[early_stopping, reduce_lr])
```

3 Callbacks

4 Mixed precision

ВЕСА КЛАССОВ

Моменты из кода

Архитектура модели 1

Веса классов 2

Исправление №3

```
early_stopping =
keras.callbacks.EarlyStopping(monitor='val_loss', patience=7, restore_best_weights=True)
reduce_lr =
keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=4, min_lr=1e-6)
```

3 Callbacks

4 Mixed precision

CALLBACKS

Моменты из кода

Архитектура модели 1

Веса классов 2

Исправление №4

```
from tensorflow.keras import
mixed_precision
mixed_precision.set_global_policy('
mixed_float16')
```

3 Callbacks

4 Mixed precision

MIXED PRECISION

Заключение 04

Подведение итогов и процесса работы

Результаты и итоги практики

В ходе практики был реализован полный цикл построения модели классификации изображений с использованием архитектуры MobileNetV2. Были проведены все ключевые этапы — подготовка и интеграция данных из различных источников, настройка аугментаций, обучение модели и её тонкая настройка (fine-tuning), что в итоге позволило достичь точности 71.64% на тестовой выборке.

Результаты и итоги практики

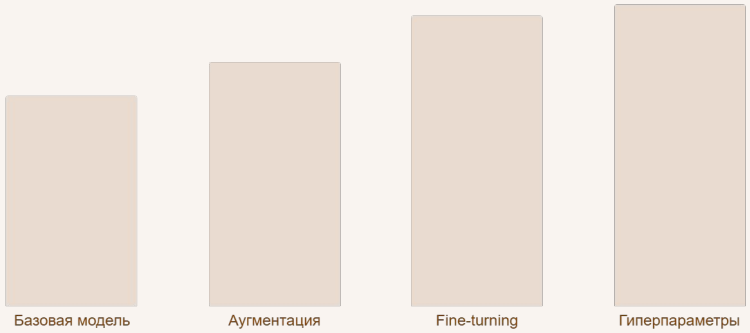
Ключевые выводы:

- Разнообразие данных критично для качества модели;
- Аугментации значительно повышают её устойчивость;
- Правильный выбор гиперпараметров влияет на стабильность обучения;
- Комплексный подход обеспечивает воспроизводимость и практическую применимость результатов.

Статистика практической работы

Потрачено дней	18
Оценка модели	5
Оценка команды	5

Влияние этапов на итоговую точность модели



Уровень сложности: умеренный, выше среднего

Низкий | Средний | Высокий

Категории 1 набора данных	Классика, модерн, ретро
Категории 2 набора данных	Лофт, эко, хай-тек

Рабочий план-график

ЭТАП №	дни	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1-Инструктаж обучающегося	1																		
2-Сборка датасета	3		Dataset "Chandeliers"																
3-Обучение модели, первая часть	5					Attempt_number_1													
4-Обучение модели, вторая часть	6										Attempt_number_2								
5-Подготовка	3																Отчёт/Презентация		

СПАСИБО
ЗА ВНИМАНИЕ!